区间和

2026年4月16日 16:03

$O(n) + O(q)$

一维

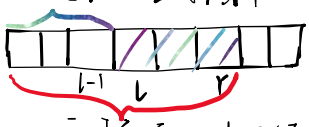
tips：从1开始

arr	7	1	1	3	2	6
idx:	1	2	3	4	5	6
dp	7	8	9	12	14	20

$\longrightarrow dp[i] = dp[i-1] + arr[i]$

$[1, i-1]$区间和 + 新元素

dp[i]：[1,i]区间元素和



[l,r]区间的和：dp[r] - dp[l-1]

区间和开n+1从1开始计数，省事

[【模板】前缀和 牛客题霸 牛客网](#)

思路：1. 构建前缀和数组

2. 使用前缀和数组找目标区间的和即可

sum[i] = sum[i- 1] + arr[i]

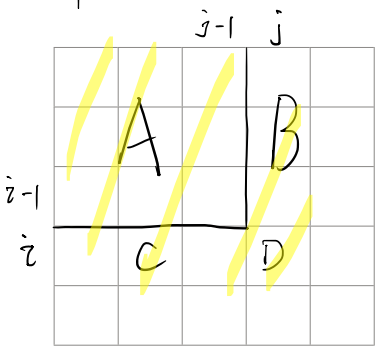
注意：前缀和数组用long long，防止溢出

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n = 0, m = 0;
    cin >> n >> m;
    // 1. 输入数组
    vector<int> arr(n + 1, 0);  // 索引0位置初始化成0，从1开始走就行
    for(int i = 1; i < n + 1; i++) {
        cin >> arr[i];
    }
    // 2. 前缀和数组，long long防止加和越界
    vector<long long> sum(n + 1, 0);
    for(int i = 1; i < n + 1; i++) {
        sum[i] = sum[i - 1] + arr[i];
    }
    // 3. 找区间答案
    int l = 0, r = 0;
    while(m--) {
        cin >> l >> r;
        cout << sum[r] - sum[l - 1] << endl;
    }
    return 0;
}
```

$O(m \cdot n) + O(q)$ → 询问次数

二维

dp[i][j]：从[1,1]到[i,j]所有的和



$$dp[i][j] = A + B + C + D \quad arr[i][j]$$

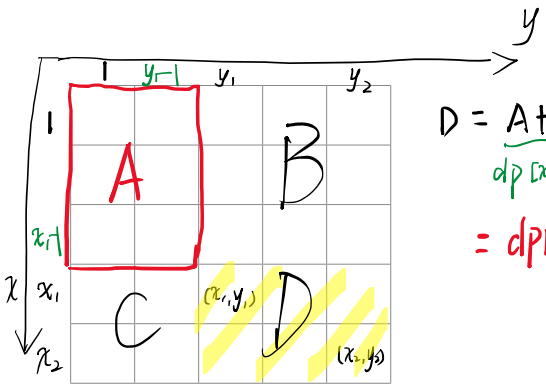
$$= \underbrace{(A+B)}_{dp[i-1][j]} + \underbrace{(A+C)}_{dp[i][j-1]} + D - \underbrace{A}_{dp[i-1][j-1]}$$

$$= dp[i-1][j] + dp[i][j-1] + arr[i][j] - dp[i-1][j-1]$$

$[x_1, y_1]$ ~ $[x_2, y_2]$ 的和

↓

求D面积



$$D = \underbrace{A+B+C+D}_{dp[x_2][y_2]} - \underbrace{(A+B)}_{dp[x_1-1][y_2]} - \underbrace{(A+C)}_{dp[x_1][y_1-1]} + \underbrace{A}_{dp[x_1-1][y_1-1]}$$

$$= dp[x_2][y_2] - dp[x_1-1][y_2] - dp[x_2][y_1-1] + dp[x_1-1][y_1-1]$$

[【模板】二维前缀和 牛客题霸 牛客网](#)

```
#include <iostream>
#include <vector>
using namespace std;
int main () {
    int n = 0, m = 0, q = 0;
    cin >> n >> m >> q;
    // 1. 构造原始输入矩阵
    vector<vector<int>> arr(n + 1, vector<int>(m + 1));
    for(int i = 1; i < n + 1; i++) {
        for(int j = 1; j < m + 1; j++) {
            cin >> arr[i][j];
        }
    }
    // 2. 构造前缀和矩阵
    vector<vector<long long>> sum(n + 1, vector<long long>(m + 1));
    for(int i = 1; i < n + 1; i++) {
        for(int j = 1; j < m + 1; j++) {
            sum[i][j] = sum[i][j - 1]
                      + sum[i - 1][j]
                      - sum[i - 1][j - 1]
```

```
                + arr[i][j];
            }
        }
        // 3. 根据区间找答案
        int x1 = 0, y1 = 0, x2 = 0, y2 = 0;
        while(q--) {
            // 一定不要搞错顺序
            cin >> x1 >> y1 >> x2 >> y2;
            long long ans = sum[x2][y2]
                          - sum[x1 - 1][y2]
                          - sum[x2][y1 - 1]
                          + sum[x1 - 1][y1 - 1];
            cout << ans << endl;
        }
    }
```

724. 寻找数组的中心下标 - 力扣（LeetCode）

1, 7, 3, 6, 5, 6

i-1 i i+1

pre[i]: pre[0] = 0，左边无
pre[1] = pre[0] + nums[1] = 1
pre[2] = pre[1] + nums[1] = 8

back[i]: back[5] = 0，右边无
back[4] = back[5] + nums[5] = 6

[0, i-1] pre[i]：[0, i-1] 区间的前缀和

[i+1, n-1] back[i]：[i+1, n-1] 区间的后缀和

1. 维护前缀、后缀和数组
2. 遍历原数组 找索引

```
class Solution {
public:
    int pivotIndex(vector<int>& nums) {
        int n = nums.size();
        vector<int> pre(n), back(n);
        for(int i = 1; i < n; i++) {
            pre[i] = pre[i - 1] + nums[i - 1];
        }
        for(int i = n - 2; i >= 0; i--) {
            back[i] = back[i + 1] + nums[i + 1];
        }
        for(int i = 0; i < n; i++) {
            if(pre[i] == back[i]) return i;
        }
        return -1;
    }
};
```

238. 除了自身以外数组的乘积 - 力扣（LeetCode）

1 2 3 4

24 12 8 6

同理，

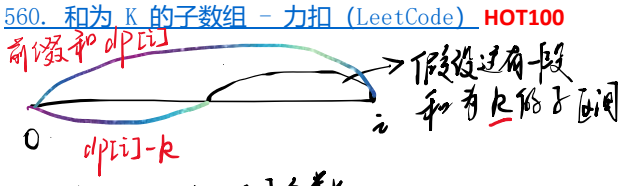
pre[i]，[0, i-1]前缀积

back[i] [i+1, n-1] 后缀积

这里pre[0] = 1，back[n-1] = 1

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        int n = nums.size();
        vector<int> pre(n), back(n), ans(n);
        pre[0] = 1, back[n - 1] = 1;
        for(int i = 1; i < n; i++) {
            pre[i] = pre[i - 1] * nums[i - 1];
        }
        for(int i = n - 2; i >= 0; i--) {
            back[i] = back[i + 1] * nums[i + 1];
        }
        for(int i = 0; i < n; i++) {
            ans[i] = pre[i] * back[i];
        }
        return ans;
    }
};
```

560. 和为 K 的子数组 - 力扣（LeetCode） HOT100



假设这有一段和为k的子区间

→ 找 dp[i]-k 子区间个数

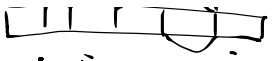
Q1：有必要再开数组吗？不！

这和暴力有什么区别？

优化查找，用 hash<int, int>

用变量表示前缀和前一位的状态

```
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        int n = nums.size();
        unordered_map<int, int> hash;
        hash[0] = 1;
        int ans = 0;
        int sum = 0;
        for(int x : nums) // 1个1个更新查找
            sum += x;
            if(hash.count(sum - k)) ans += hash[sum - k];
```



优化查找，用 hash[<int, int>] 中间 = k → 下标

用变量表示前缀和 前一位的状态

Q2：

到[0,i]找，不行！

$\boxed{hash[0] = 1}$

从数组开头到当前位置的整个子数组和为 k

```
    int sum = 0;
    for(int x : nums) 什么更新查找
        sum += x;
        if(hash.count(sum - k)) ans += hash[sum - k];
        hash[sum]++;
    }
    return ans;
}
};
```

974. 和可被 K 整除的子数组 - 力扣（LeetCode）

补充：C++中负数取模　负%b → 负

纠正负数：$(a\%b+b)\%b$

剩余思路同上 ~

```
class Solution {
public:
    int subarraysDivByK(vector<int>& nums, int k) {
        int ans = 0;
        unordered_map<int, int> hash;
        hash[0 % k] = 1;
        int sum = 0;
        for(int e : nums) {
            sum += e;
            int r = (sum % k + k) % k;
            if(hash.count(r)) ans += hash[r];
            hash[r]++;
        }
        return ans;
    }
};
```

525. 连续数组 - 力扣（LeetCode）

只有0和1，把0全变为-1

和为0的最长连续子数组

```
class Solution {
public:
    int findMaxLength(vector<int>& nums) {
        int n = nums.size(), ans = 0;
        for(auto& x : nums) {
            if(0 == x) x = -1;
        }
        unordered_map<int, int> hash;
        hash[0] = -1;
        int sum = 0;
        for(int i = 0; i < n; i++) {
            sum += nums[i];
            if(hash.count(sum)) ans = max(ans, i - hash[sum]);
            else hash[sum] = i;
        }
        return ans;
    }
};
```